

ZDROVENA

Unified Implementation & Release Plan

FastAPI + Azure + React SPA

Cel dokumentu

Jeden repo-ready plan łączący master plan, wykonane prace R0-R3, plan R4-R6, kryteria akceptacji, bramy jakości i kolejność wdrożeń. Dokument nie zakłada przepisywania systemu - rozwój pozostaje punktowy, atomowy i operacyjnie bezpieczny.

Właściciel: Piotr Gryzłó

Repozytorium: PiotrGry/zdrovena-reconciliation

Wersja planu: 1.0

Data: 12.07.2026

1. Streszczenie wykonawcze

Zdrovena-reconciliation jest dojrzałym wewnętrznym systemem back-office. Fundamenty techniczne są solidne: FastAPI, Azure Container Apps, React SPA, walidacja JWT/HMAC, Decimal w logice finansowej, optimistic concurrency w Table Storage, rozbudowany zestaw testów backendowych i wielowarstwowe CI.

Największym obecnym ryzykiem nie jest brak funkcji, lecz wiarygodność sygnałów jakości. Zielony workflow nie zawsze oznacza, że pełny uwierzytelniony flow działa. Dlatego plan skupia się kolejno na: domknięciu obserwowalności i smoke testów, fakturowaniu/kaucji, state machine przesylek, etykietach, kontrakcie FE-BE i testach E2E.

Zasada nadrzędna

Nie testujemy na produkcji. Każdy release przechodzi przez staging, smoke strict, autoryzowany test API i ręczną weryfikację krytycznego flow. Produkcja jest etapem promocji, nie pierwszym miejscem walidacji.

1.1 Stan realizacji

Release	Status	Najważniejsze elementy	Ocena
R0	Wdrożone	Zdjęcie xfail, fix Azure OIDC permissions	Zamknięte
R1	Wdrożone	fetchJson, toasty, ErrorBoundary, koperta błędów, realny progress	Zamknięte
R2	Wdrożone z follow-upami	Correlation ID, eventy, auth guard, CORS, release validation	Wymaga utwardzenia
R3	Wdrożone z weryfikacją infra	Polling, widok DLQ, action group, alert DLQ	Wymaga apply i testu alertu
R4	Następny	Fakturowanie, kaucja, pętla 502, preview = faktyczna faktura	Krytyczny biznesowo
R5	Planowany	State machine, atomic claim, etykiety pojedyncze i zbiorcze	Średni blast radius
R6	Planowany	Testy frontendowe, kontrakty, fake providers, E2E	Domknięcie jakości

2. Zasady realizacji

- Każda jednostka wdrożeniowa jest atomowa i zostawia develop zielone.
- Jeden commit na jednostkę, Conventional Commits.
- Brak zmian w logice KSeF / Rozliczenia / fakturowania poza jawnie wskazanym zakresem.
- Zmiany krytyczne biznesowo są wdrażane przy niskim wolumenie i z ręczną walidacją.
- Staging musi używać prawdziwego auth, ról i konfiguracji zbliżonej do produkcji.

- Mockujemy granice zewnętrzne, nie własne moduły domenowe.
- Nowe funkcje operacyjne muszą mieć: log sukcesu, log błędu, correlation ID, test i kryterium rollbacku.

2.1 Definition of Done

- Kod, testy i dokumentacja zmienione w tym samym commicie lub pakiecie.
- Backend: ruff, ruff format, pyright, pytest z wymaganym coverage.
- Frontend: lint, build i - po R6 - testy komponentowe.
- Smoke strict na stagingu bez krytycznych SKIP.
- Brak authenticated 401/403 w krytycznych testach.
- Logi zawierają correlation ID i zdarzenie sukcesu/porażki.
- Rollback jest możliwy przez poprzednią rewizję Container Apps lub revert commita.

3. Harmonogram release'ów

Release	Develop -> main	Zakres	Prod	Weryfikacja
R0	15.07	PR-1 + fix OIDC	Formalny	CI zielone na main.
R1	02.08	PR-2...PR-7 - pakiet feedbacku	03.08 rano	Spinner i polski toast po sztucznym błędzie.
R2	09.08	PR-8, PR-9, PR-12	10.08 rano	Webhook i draft spięte jednym ID.
R3	16.08	PR-10, PR-17; PR-16 osobno	17.08	Draft widoczny <=20 s; testowy alert dociera.
R4	23.08	PR-11 + PR-13/27 + PR-14	25.08 rano	3-5 faktur zgodnych co do grosza; pętla 502 zamknięta.
R5	30.08	PR-15, PR-18, PR-20 + PR-25/26	31.08	LABEL_NOT_READY i bulk print działają.
R6	06.09	PR-19 + test foundation	Formalny	Wiarygodne contract/E2E gates.

Przyspieszenie

R0-R3 są już funkcjonalnie wykonane. Realny harmonogram może zostać skrócony, jeżeli najpierw domkniemy wiarygodność CI/staging, a potem uruchomimy krótkie, niezależne pakiety R4.1-R4.4 i R5A-R5B. Przyspieszenie oznacza krótsze cykle, nie większy blast radius.

4. Status R0-R3 i niezbędne follow-upy

R0 - odblokowanie procesu

- Usunięto nieaktualny xfail(strict=True).
- Dodano wymagane permissions dla Azure OIDC.
- Develop pozostaje zielone.

R1 - feedback operatora

- fetchJson z polską kopertą błędów i bez zależności od pustego statusText.
- Globalne toasty, obsługa błędów akcji, ErrorBoundary.
- RunPanel pokazuje realne completed_steps zamiast sztucznej animacji.

R2 - observability i bezpieczeństwo

- Correlation ID w odpowiedzi, logach, kopercie błędu i zadaniach tła.
- Strukturalne eventy sukcesu.
- Guard przed AZURE_AUTH_DISABLED na produkcji.
- CORS rozszerzony o PATCH/OPTIONS.

R3 - reaktywność i DLQ

- Polling 20 s, visibility-aware.
- Widok DLQ z retry i discard.
- Action group i alert DLQ w Terraformie.

4.1 Follow-up R2/R3 - pakiet konsolidacyjny

Element	Zmiana	Kryterium akceptacji
Smoke strict	Brak tokena lub krytyczny SKIP musi kończyć release validation błędem.	0 krytycznych SKIP.
Authenticated 401/403	Każdy 401/403 po wysłaniu tokena jest FAIL, nigdy SKIP.	Test regresyjny + zielony staging.
Warm-up	Brak HTTP 200 po retry kończy workflow błędem.	Martwe API nie może dać zielonego release.
Correlation ID	Walidacja formatu i długości; reset ContextVar w finally.	Test regresyjny + zielony staging.
APP_ENV	Jedna obowiązkowa zmienna, fail-closed dla produkcji.	Test regresyjny + zielony staging.
PII	Maskowanie e-maili, trackingów i identyfikatorów w logach strukturalnych.	Test regresyjny + zielony staging.
Monitoring	Terraform apply, weryfikacja nazwy tabeli KQL, testowy alert i potwierdzenie maila.	Alert pojawia się w Azure Monitor i dociera e-mailem.

5. R4 - fakturowanie i kaucja

R4 jest następnym priorytetem biznesowym. Zmiany powinny być podzielone na cztery małe jednostki, aby ograniczyć ryzyko księgowe i umożliwić osobne rollbacki.

R4.1 - pętla 502

Odzyskanie istniejącej faktury i dokończenie procesu zamiast resetu ID oraz ponownego tworzenia.

R4.2 - jedno źródło kaucji

Wspólna funkcja kalkulacyjna; preferencja natywnego deposit z Allegro, jawny fallback.

R4.3 - idempotencja

Guard wielu faktur, re-read przed zapisem, poprawna obsługa quantity=0, Decimal end-to-end.

R4.4 - preview zgodny z rzeczywistością

VAT z prawidłowego pola, settlement_positions, porównanie do totalToPay bez dostawy.

5.1 Testy obowiązkowe R4

- Prawdziwy zanonimizowany payload Allegro -> mapper -> endpoint -> response.
- Nie mockować allegro_order_to_fakturownia_invoice w testach endpointu.
- Wiele faktur dla jednego zamówienia nie powoduje wielokrotnego dodania pełnej kaucji.
- quantity=0 jest walidowane zgodnie z regułą biznesową.
- Kwoty finansowe pozostają Decimal do granicy serializacji.
- Retry po częściowym sukcesie nie tworzy drugiej faktury.

5.2 Weryfikacja po wdrożeniu R4

Order ID	totalToPay	Dostawa	Produkty	Kaucja	Faktura brutto	Settlement	Różnica

Warunek akceptacji: różnica = 0,00 PLN dla każdej sprawdzonej faktury.

6. R5 - state machine przesyłek i etykiety

6.1 R5A - bezpieczeństwo realizacji draftu

- Atomowy claim przed execute_draft.
- Blokada double execute i duplikacji przesyłek.
- Jawna state machine dozwolonych przejść.
- Guard mark_fulfilled dla statusów cancelled/error.
- Retry bez ponownego side effectu u kuriera.

6.2 R5B - drukowanie etykiet

- LABEL_NOT_READY (HTTP 409) dla niepotwierdzonej przesyłki InPost.
- Toast informacyjny zamiast ogólnego 502/alert().
- POST /shipping/labels/batch grupujący drafty po kurierze.
- InPost/Allegro bulk labels, Apaczka przez turn_in lub scalanie PDF.
- Opcjonalny wybór A6/A4 i PDF/ZPL.

6.3 Kryteria akceptacji R5

- Dwa równoległe execute dla tego samego draftu tworzą maksymalnie jedną przesyłkę.
- Etykieta przed confirmed daje czytelny komunikat, nie 502.
- Druk zaznaczonych draftów zwraca jeden poprawny PDF.
- Retry i rollback pozostawiają spójny status draftu.

7. R6 - domknięcie jakości i development bez produkcji

R6 nie powinien być jedynym momentem dodawania testów. Testy regresyjne powstają razem z R4/R5, natomiast R6 buduje trwałą infrastrukturę testową.

7.1 Frontend

- Vitest + React Testing Library.
- Testy ShippingView, DLQ, error envelope, polling i adresów.
- Generowanie typów TypeScript z FastAPI OpenAPI.
- CI wykrywa niezaktualizowany kontrakt.

7.2 Fake providers

- Fake Allegro, InPost, Apaczka i Fakturownia po HTTP.
- Walidacja method, URL, headers, body i enumów.
- Stateful scenariusze: retry, timeout, 500, duplikacja i partial success.
- Fixture jest danymi wejściowymi, nie gotową odpowiedzią z mocka.

7.3 E2E

1. Webhook -> draft -> widoczność w UI.
2. Draft -> realizacja InPost -> tracking.
3. Provider 500 -> polski toast + correlation ID.
4. Invoice preview -> create invoice -> zgodność kwot.
5. DLQ -> retry -> draft utworzony.

8. Docelowy pipeline CI/CD

Etap	Co sprawdza	Warunek przejścia
1. PR quality gate	Ruff, format, Pyright, pytest, security, frontend lint/build/test.	Wszystkie wymagane kroki zielone; 0 krytycznych SKIP.
2. Build	Obraz Docker + lokalny health/ready test.	Wszystkie wymagane kroki zielone; 0 krytycznych SKIP.
3. Deploy staging	Nowa rewizja Container Apps i SWA.	Wszystkie wymagane kroki zielone; 0 krytycznych SKIP.
4. Staging validation	Warm-up, strict auth smoke, business dry-run, frontend smoke.	Wszystkie wymagane kroki zielone; 0 krytycznych SKIP.
5. Approval	Ręczna akceptacja dla zmian finansowych i shipping side effects.	Wszystkie wymagane kroki zielone; 0 krytycznych SKIP.
6. Promocja prod	Deploy przy niskim wolumenie.	Wszystkie wymagane kroki zielone; 0 krytycznych SKIP.

7. Post-deploy	Kontrolne logi, alerty i 3-5 ręcznych przypadków.	Wszystkie wymagane kroki zielone; 0 krytycznych SKIP.
----------------	---	---

9. Rejestr ryzyk

Ryzyko	Wpływ	Priorytet	Mitigacja
Fałszywie zielony smoke	Wysokie	P0	Strict mode; required tests; authenticated 401 = FAIL.
Błąd księgowy kaucji	Wysokie	P0	Jedno źródło kalkulacji; Decimal; ręczna kontrola 3-5 faktur.
Double execute przesyłki	Wysokie	P1	Atomowy claim i state machine.
Alert Terraform nie działa	Średnie	P1	Apply + testowy alert + potwierdzenie e-mail.
Rozjazd FE-BE	Średnie	P1	OpenAPI generated types + contract tests.
Sandbox providera zawodny	Średnie	P1	Własny fake provider HTTP.
Duży webhooks.py	Niskie krótkoterminowo	P3	Refaktor po stabilizacji R0-R6.

10. Rollback i operacyjna kontrola

- Każdy release ma osobny commit lub małą serię commitów.
- Container Apps: powrót do poprzedniej rewizji.
- Frontend SWA: rollback do poprzedniego artefaktu/builda.
- Zmiany finansowe: feature flag lub wyłączenie automatycznego patchowania.
- State machine: możliwość ręcznego odblokowania draftu przez kontrolowany endpoint/operator flow.
- Terraform: plan przed apply, osobny commit i brak łączenia z logiką aplikacji.

11. Kolejność implementacji od teraz

1. **Pakiet A - konsolidacja R2/R3:** Smoke strict, warm-up fail, correlation cleanup, auth fail-closed, weryfikacja alertów.
2. **Pakiet B - R4.1/R4.2:** Pętla 502 i wspólna kalkulacja kaucji.
3. **Pakiet C - R4.3/R4.4:** Idempotencja i prawdziwy invoice preview.
4. **Pakiet D - R5A:** Atomic claim i state machine.
5. **Pakiet E - R5B:** LABEL_NOT_READY i bulk labels.
6. **Pakiet F - R6:** Frontend tests, OpenAPI contracts, fake providers, E2E.

12. Checklisty

12.1 Przed merge do main

- ☐ Zakres commita jest atomowy.
- ☐ Testy lokalne/CI zielone.
- ☐ Brak krytycznych SKIP.
- ☐ Zmiana ma kryterium akceptacji i rollback.
- ☐ Dokumentacja/env template zaktualizowane.

12.2 Przed deployem produkcyjnym

- ☐ Staging deploy zakończony.
- ☐ Smoke strict przeszedł.
- ☐ Authenticated endpoints zwracają oczekiwane statusy.
- ☐ Logi i correlation ID widoczne.
- ☐ Alert testowy działa.
- ☐ Okno niskiego wolumenu potwierdzone dla R4/R5.

12.3 Po deployu

- ☐ Health/ready 200.
- ☐ Brak skoku 5xx/latency.
- ☐ Pierwsze realne operacje zweryfikowane.
- ☐ Brak wpisów w DLQ bez reakcji.
- ☐ Rollback pozostaje dostępny do zamknięcia obserwacji.

13. Źródła planu

- zdrovena_master_plan.pdf
- zdrovena_r0_r1_summary.pdf
- zdrovena_gate_r2_summary.pdf
- zdrovena_r3_summary.pdf
- zdrovena_label_printing_plan.pdf
- Review PR #116, testów i logów GitHub Actions wykonane 12.07.2026.

Status dokumentu

Plan jest dokumentem żywym. Po każdym release należy zaktualizować status, hash końcowego commita, wyniki testów, obserwacje po wdrożeniu i decyzję o przejściu do kolejnego pakietu.

14. Instrukcje wykonawcze dla GitHub Copilot

Ta sekcja jest obowiązującym kontraktem wykonawczym. Copilot może implementować zadania wyłącznie pojedynczo, zgodnie z zakresem issue. Główny plan opisuje kierunek; issue opisuje dokładną jednostkę wdrożeniową.

- Pracuj tylko nad jednym issue naraz.
- Nie modyfikuj plików i funkcji poza jawnym zakresem, chyba że bez tego kod nie kompiluje się; każdą taką zmianę opisz.
- Nie zmieniaj logiki KSeF, miesięcznego Rozliczenia ani fakturowania poza zakresem konkretnego issue.
- Nie osłabiaj, nie usuwaj i nie oznaczaj jako skip/xfail testów tylko po to, aby CI przeszło.
- W kodzie finansowym używaj Decimal; nie wprowadzaj float do obliczeń pieniężnych.
- W testach integracyjnych nie mockuj wewnętrznych mapperów i reguł domenowych. Mockuj tylko granice zewnętrzne.
- Każdy zewnętrzny side effect musi być idempotentny albo chroniony atomowym claimem.
- Nigdy nie wykonuj write-call do produkcyjnego Allegro, InPost, Apaczki lub Fakturowni.
- Nie używaj sekretów produkcyjnych w testach.
- Nie testuj na produkcji. Walidacja odbywa się lokalnie i na stagingu.
- Każdy błąd z uwierzytelnionego smoke requestu jest FAIL, nie SKIP.
- Każda zmiana musi mieć test regresyjny, kryterium akceptacji i sposób rollbacku.
- Przed zakończeniem podaj: zmienione pliki, powód zmian, komendy walidacyjne i nierozwiązane ryzyka.

Warunek zatrzymania

Copilot powinien zatrzymać implementację i opisać blokadę, gdy reguła biznesowa jest niejednoznaczna, wymagany jest produkcyjny sekret lub realny write-call, potrzebna jest migracja danych bez rollbacku, albo zakres zaczyna naruszać logikę KSeF/Rozliczenia poza issue.

14.1 Format odpowiedzi Copilota po wykonaniu issue

- Podsumowanie root cause.
- Lista zmienionych plików.
- Opis zachowania przed i po zmianie.
- Lista dodanych/zmienionych testów.
- Dokładne komendy uruchomione lokalnie.
- Wyniki testów i coverage.
- Ryzyka nadal niezwerifikowane.
- Proponowany commit message.

15. Szablon pojedynczego GitHub Issue

Sekcja issue	Wymagana zawartość
Goal	Jednoznaczny efekt biznesowy lub techniczny.
Current behavior	Jak system zachowuje się dziś, wraz z konkretnym failure mode.
Expected behavior	Dokładny stan po implementacji.
Scope	Pliki, funkcje i endpointy do zmiany.

Out of scope	Elementy, których nie wolno dotykać.
Implementation notes	Wskazówki, nie gotowe obejście testów.
Tests	Przypadki pozytywne, negatywne, retry i concurrency.
Acceptance criteria	Mierzalna checklista.
Validation commands	Komendy lint/typecheck/test/build.
Rollback	Jak bezpiecznie wycofać zmianę.
Copilot guardrails	Reguły specyficzne dla zadania.

16. Backlog GitHub Issues

Poniższy backlog jest gotowy do utworzenia jako GitHub Issues. Kolejność jest celowa: najpierw wiarygodność stagingu, potem ryzyka finansowe, state machine, etykiety i fundament testów.

Nr	Tytuł	Zależność	Release
1	R4-A: make staging smoke validation strict and eliminate false-green 401/SKIP results	brak	R4 consolidation
2	R4-B: harden correlation ID propagation and fail-closed production auth configuration	R4-A	R4 consolidation
3	R4-C: verify Azure monitoring delivery and make DLQ alert operational	R4-B	R4 consolidation
4	R4.1: recover partially created Allegro invoices instead of looping on HTTP 502	R4-A	R4
5	R4.2: unify Allegro deposit calculation and keep monetary values Decimal-safe	R4.1	R4
6	R4.3: make invoice preview match the final invoice and Allegro totalToPay	R4.2	R4
7	R5-A: add atomic draft execution claim and enforce shipment state transitions	R4-A	R5

8	R5-B: add LABEL_NOT_READY handling and batch label printing	R5-A	R5
9	R6: add frontend tests, generated API contracts, fake providers and critical E2E flows	R4/R5 testy regresyjne	R6

16.1 Zasada pracy z backlogiem

- Issue jest źródłem prawdy dla zakresu konkretnej implementacji.
- Copilot nie powinien realizować kilku issue w jednym PR.
- Każdy PR linkuje dokładnie jedno issue i używa checklisty akceptacji.
- Po merge issue jest zamykane dopiero po staging validation.
- Zmiany Terraform są wdrażane i weryfikowane oddzielnie od zmian aplikacji.